

Lab 3: Analysis of parallel strategies. The computation of the Mandelbrot set

Parallelism

Universitat Politècnica de Catalunya (UPC)

Facultat d'Informàtica de Barcelona (FIB)

Authors:

FERRAN MESAS

`ferran.mesas@estudiantat.upc.edu`

ISHAK FELFOUL

`ishak.felfoul@estudiantat.upc.edu`

April 17, 2026

Table of Contents

1	Iterative Decomposition Strategies	2
1.1	Original tile parallel strategy without options	3
1.2	Original tile parallel strategy with -d option	5
1.3	Original tile parallel strategy with -h option	10
1.4	Fine grain parallel strategy	15
1.5	Column of tiles parallel strategy	23
2	Recursive Decomposition Strategies	26
2.1	Leaf recursive parallel strategy	27
2.2	Tree recursive parallel strategy	29
3	Final Strategy Comparison	34
3.1	Comparison table	34
3.2	Conclusions	34

1. Iterative Decomposition Strategies

This section presents the iterative parallelization strategies analyzed for the Mandelbrot program. In particular, we study the original tiled decomposition, a fine-grain version of the tiled strategy, and a column-of-tiles decomposition. In addition, for the original tiled version we also analyze the behaviour of the program when the `-d` and `-h` options are enabled, in order to study the dependencies introduced by graphical output and histogram updates.

Unless explicitly stated otherwise in the corresponding subsection, all executions were performed without additional command-line arguments. Therefore, the only subsections using extra arguments are those that explicitly analyze the `-d` and `-h` variants.

1.1. Original tile parallel strategy without options

Source code. Submitted file(s): `codes/mandel-seq-iter-tar.c`.

Task Dependency Graph

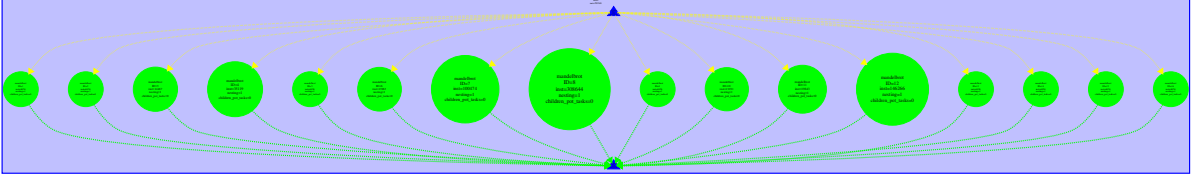


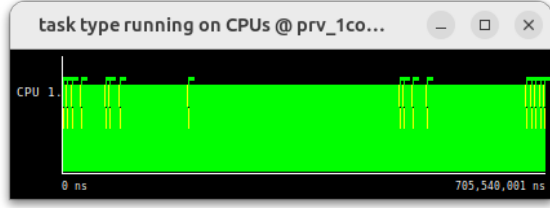
Figure 1: Task dependency graph without extra options.

Although this version has no inter-task dependencies between `mandelbrot` tasks, the Task Dependency Graph shows a strong load imbalance. The number of instructions varies significantly across tiles, ranging from 6656 instructions for the lightest tasks to 308644 instructions for task 8 and 146266 for task 12. This imbalance is explained by the structure of the algorithm itself. Each tile first checks its borders and, if all border points have the same dwell value, the whole tile is filled directly; otherwise, the task computes the full tile pixel by pixel. Moreover, the cost of each pixel depends on the number of iterations executed inside `pixel_dwell()`, which varies according to how quickly the corresponding point escapes the Mandelbrot set. As a consequence, tiles located in uniform regions are much cheaper, while tiles near the boundary of the set are considerably more expensive. The same imbalance will also appear in the `-d` and `-h` versions, since both options only add extra display or histogram work on top of the same Mandelbrot computation, and therefore introduce only a marginal overhead with respect to the variation caused by the underlying escape-iteration cost.

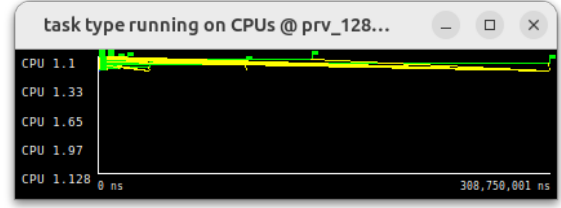
Dependency Analysis

This Task Dependency Graph exposes the maximum task-level parallelism for the original tiled decomposition, since no dependencies exist between tasks computing different Mandelbrot tiles. The only visible edges are from `MAIN_TAREADOR` to the `mandelbrot` tasks, reflecting that the main routine initializes the shared input parameters and buffers before task creation. Therefore, these edges represent setup dependencies rather than true inter-task dependencies among Mandelbrot computations.

Execution Time



(a) $T_1 = 705,540,001$ ns



(b) $T_{128} = 308,750,001$ ns

Figure 2: Execution-time simulation for the original tiled strategy without options.

The simulated sequential execution is $T_1 = 705,540,001$ ns, while the 128-core simulation gives $T_{128} = 308,750,001$ ns, which we use as an approximation of T_∞ .

1.2. Original tile parallel strategy with -d option

Source code. Submitted file(s): `codes/mandel-seq-iter-tar-option-d.c`.

Task Dependency Graph



Figure 3: Task dependency graph with option -d.

Dependency Analysis

With the `-d` option, each `mandelbrot` task not only computes its own tile of the matrix `M`, but also performs graphical output through X11. In particular, all tasks execute `XSetForeground(display, gc, color)` followed by `XDrawPoint(display, win, gc, px, py)` while sharing the same global display objects. As shown in Figure 4, Tareador identifies this shared display state as a dependency, reported in the Real dependency view under the label `X11_COLOR_fake`. Although this name does not correspond to an explicit variable in the source code, it is consistent with hidden state associated with the X11 drawing operations. This explains the almost serialized Task Dependency Graph. However, this dependency does not come from the Mandelbrot computation itself, since different tasks still write disjoint tiles of `M`. Therefore, the order is not fundamentally imposed by the algorithm, but by the side effects of the graphical output.

This dependency could be handled through data-sharing synchronization. For example, a lock could protect the display section so that each task computes its tile in parallel and only serializes the pair of X11 calls that update the shared graphical state. A cleaner alternative would be to decouple computation from display: tasks would only compute and store their results in `M`, and the graphical output would be performed afterwards by the main task (or by a dedicated display task). In that case, the Mandelbrot tile computation would remain fully parallel, and only the final rendering stage would be serialized.

For the remainder of the analysis, we assume that this dependency is handled through data-sharing synchronization. Under this assumption, the Mandelbrot computations remain parallel and the synchronization overhead associated with the display operations is neglected, since Tareador does not provide a direct way to model it.

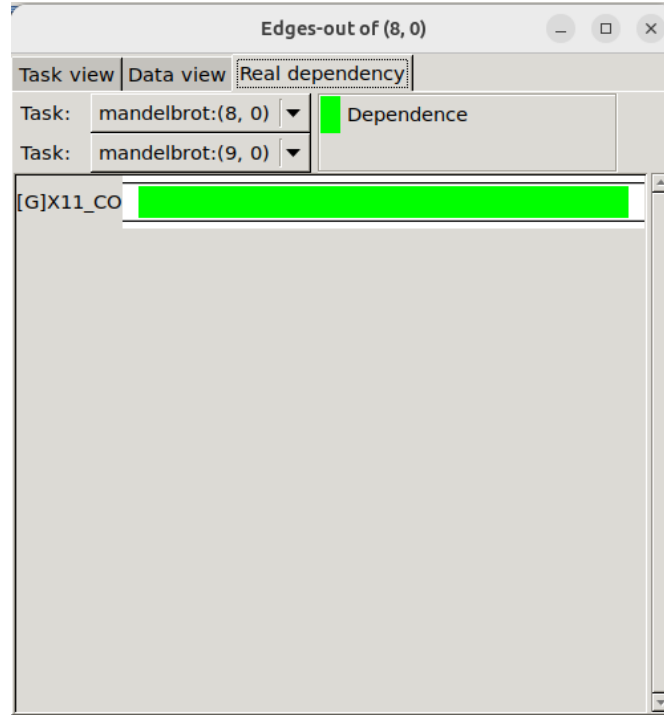


Figure 4: Real dependency view for two `mandelbrot` tasks in the `-d` execution. Tareador reports a dependency associated with the state variable `X11_COLOR_fake`, which represents the shared X11 display state involved in the graphical output.

Code Changes for the `-d` Option

```
void mandel_tiled(int maxiter, int output2display, int output2file,
    int output2histogram, int output2measurement, int * histogram) {

    int equal;

    tareador_disable_object(&X11_COLOR_fake);

    for (int y=0; y<ROWS; y+=TILE) {
        for (int x=0; x<COLS; x+=TILE) {
            tareador_start_task("mandelbrot");
            ...
        }
    }
}
```

To obtain the Task Dependency Graph without the dependencies caused by the display path, we added the call `tareador_disable_object(&X11_COLOR_fake)` before the tiled loops. This modification does not change the task decomposition or the Mandelbrot computation, it only instructs Tareador to ignore the shared X11 display state when building the dependency graph.

Task Dependency Graph Without Data-Sharing Dependencies

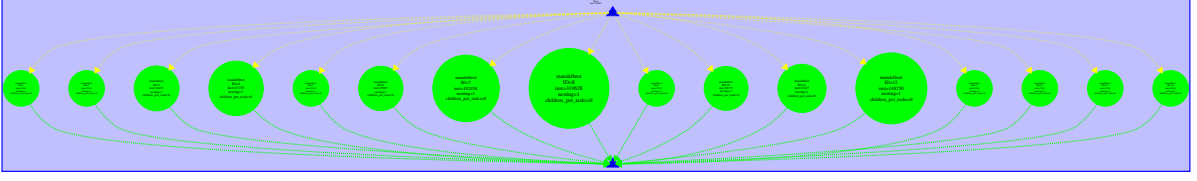
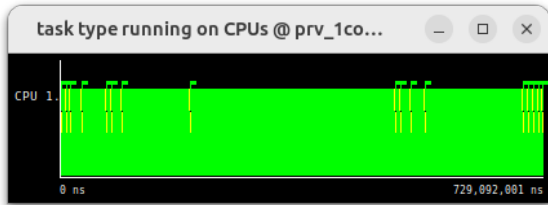


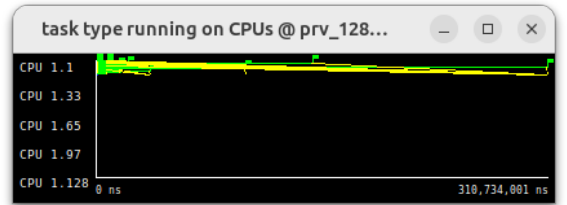
Figure 5: Task dependency graph for the `-d` option after removing dependencies caused by data sharing.

After disabling the data-sharing dependency associated with the display state, the resulting TDG has the same dependency-free structure as the baseline version. Compared with the no-option execution, each `mandelbrot` task performs more instructions because the task body still includes the X11 display operations. This increase is visible for all corresponding tasks, although its relative impact is much larger for the lighter tiles. For example, task 1 increases from 6656 to 7616 instructions (+14.4%), whereas task 8 increases from 308644 to 310628 (+0.64%). Thus, removing the dependency from Tareador’s analysis exposes the available parallelism, but it does not remove the computational cost of the display code itself.

Execution Time



(a) $T_1 = 729,092,001$ ns



(b) $T_{128} = 310,734,001$ ns

Figure 6: Execution-time simulation for the original tiled strategy with `-d`.

The simulated sequential execution is $T_1 = 729,092,001$ ns, while the 128-core simulation gives $T_{128} = 310,734,001$ ns, which we use as an approximation of T_∞ .

1.3. Original tile parallel strategy with -h option

Source code. Submitted file(s): `codes/mandel-seq-iter-tar-option-h.c`.

Task Dependency Graph

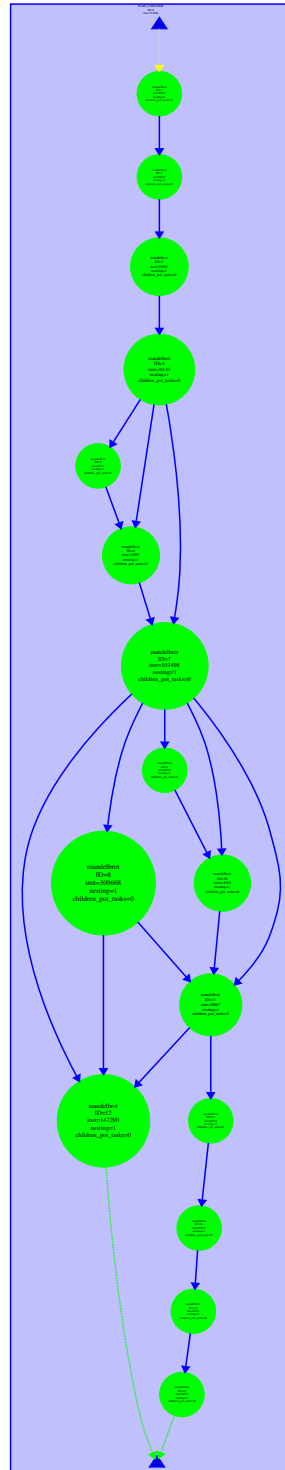


Figure 7: Task dependency graph with option -h.

Dependency Analysis

With the `-h` option, each `mandelbrot` task not only computes its own tile of the matrix `M`, but also updates the global histogram of dwell values. As shown in Figure 8, Tareador reports dependencies through a heap-allocated object, which is consistent with the shared histogram array. This is expected, since the histogram is dynamically allocated and all tasks may update the same bins. In particular, when a tile is uniform, the code performs an accumulated update of the form `histogram[M[y][x]-1] += (TILE*TILE)`; otherwise, it performs per-pixel updates of the form `histogram[M[py][px]-1]++`. Therefore, different tasks may concurrently access and modify the same histogram position, which introduces data-sharing dependencies.

However, these dependencies do not come from the Mandelbrot computation itself, since different tasks still write disjoint tiles of `M`. The loss of parallelism is due to the shared histogram updates, not to true algorithmic dependencies between tiles. Hence, the order is not fundamentally imposed by the algorithm and can be handled through data-sharing synchronization. A straightforward solution would be to make the histogram updates atomic, so that increments and accumulated additions to a shared bin are performed safely. Under this assumption, the Mandelbrot tile computations remain parallel, and only the accesses to the shared histogram are synchronized.

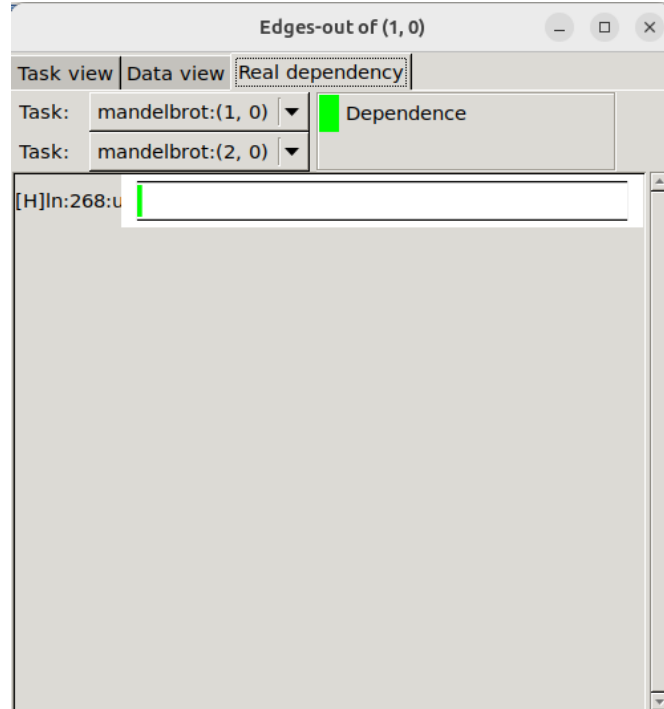


Figure 8: Real dependency view for two `mandelbrot` tasks in the `-h` execution. Tareador reports the dependency through a heap-allocated object, which is consistent with the shared histogram array.

Code Changes for the -h Option

```
void mandel_tiled(int maxiter, int output2display, int output2file,
    int output2histogram, int output2measurement, int * histogram) {

    int equal;

    tareador_disable_object(histogram);

    for (int y=0; y<ROWS; y+=TILE) {
        for (int x=0; x<COLS; x+=TILE) {
            tareador_start_task("mandelbrot");
            ...
        }
    }
}
```

To obtain the Task Dependency Graph without the dependencies caused by histogram updates, we added the call `tareador_disable_object(histogram)` before the tiled loops, notice that `histogram` is already a pointer. As in the previous case, this does not modify the task structure or the Mandelbrot computation itself, but only removes the shared histogram object from Tareador's dependency analysis.

Task Dependency Graph Without Data-Sharing Dependencies

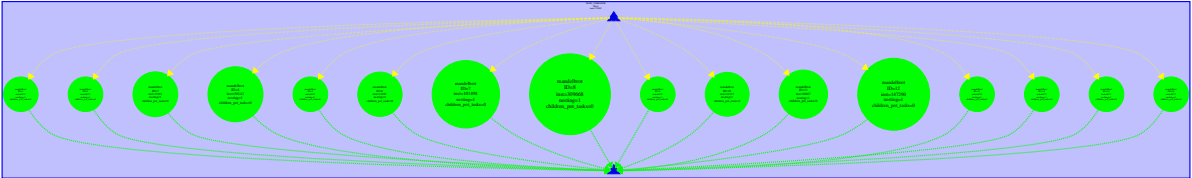
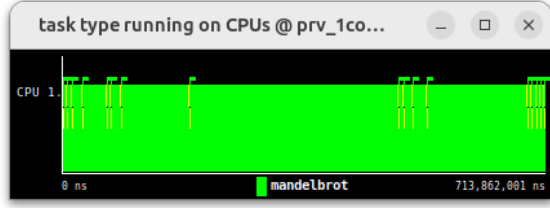


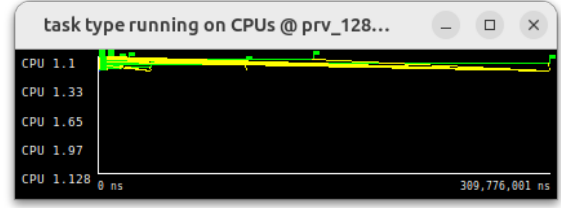
Figure 9: Task dependency graph for the `-h` option after removing dependencies caused by data sharing.

After disabling the data-sharing dependency associated with the histogram, the resulting TDG also matches the dependency-free structure of the baseline version. Compared with the no-option execution, each `mandelbrot` task performs slightly more instructions because the task body still includes the histogram updates. Again, this increase appears in all corresponding tasks, but its relative effect is more noticeable for the lighter ones. For instance, task 1 increases from 6656 to 6672 instructions (+0.24%), whereas task 8 increases from 308644 to 309668 (+0.33%). Therefore, disabling the histogram dependency only changes the dependency analysis, the extra work associated with maintaining the histogram remains present in the task body.

Execution Time



(a) $T_1 = 713,862,001$ ns



(b) $T_{128} = 309,776,001$ ns

Figure 10: Execution-time simulation for the original tiled strategy with `-h`.

The simulated sequential execution is $T_1 = 713,862,001$ ns, while the 128-core simulation gives $T_{128} = 309,776,001$ ns, which we use as an approximation of T_∞ .

1.4. Fine grain parallel strategy

Source code. Submitted file(s): `codes/mandel-seq-iter-tar-fine-grain.c` and `codes/mandel-seq-iter-tar-fine-grain-no-data-sharing.c`. The second file corresponds to the version used to remove data-sharing dependencies in the TDG analysis.

Task Dependency Graph

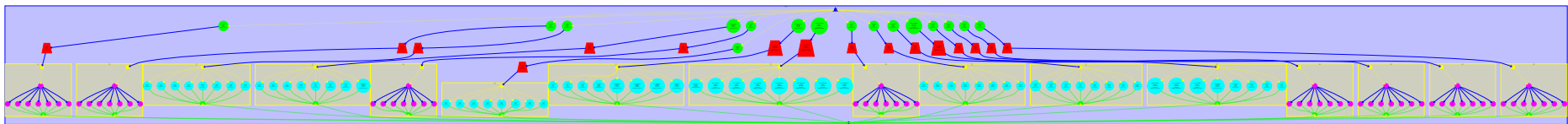


Figure 11: Task dependency graph for the fine-grain parallel strategy.

In the fine-grain strategy, each 8×8 tile is split into three first-level tasks: `check_horizontal`, `check_vertical`, and `tile_filling`. Since the image contains $4 \times 4 = 16$ tiles, this produces 48 first-level tasks. Moreover, each `tile_filling` task generates 8 second-level tasks, one per row of the tile, which are either `tile_copy_borders` or `tile_computation`. As a result, the Task Dependency Graph is substantially richer than in the original tiled version.

A clear load imbalance is also visible: `tile_copy_borders` have a small almost fixed cost, while `check_horizontal`, `check_vertical`, and especially `tile_computation` have variable cost because they call `pixel_dwell()`, whose iteration count depends on the position of each point in the Mandelbrot set.

Dependency Analysis

In the fine-grain strategy, the relevant dependencies come from two shared values. As shown in Figure 12, `check_horizontal` and `check_vertical` are dependent because both contribute to the shared variable `equal`, which is later used to determine whether the tile is uniform.

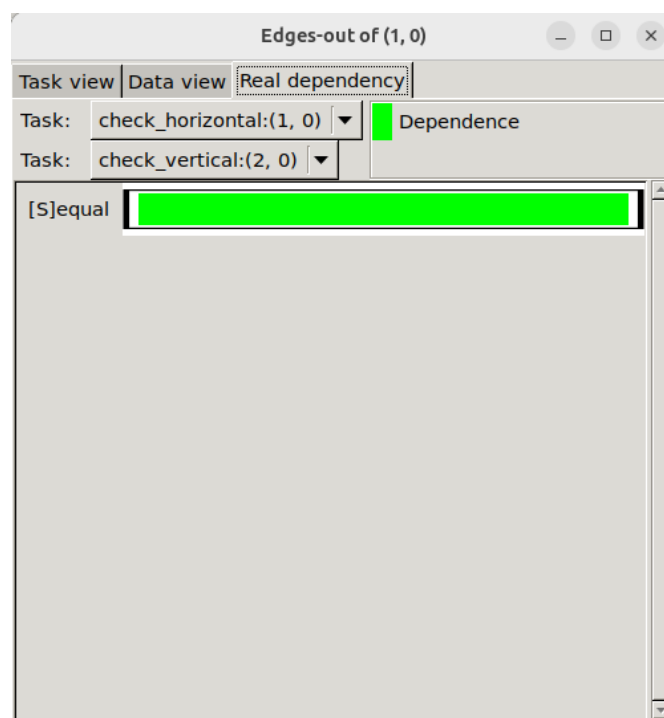


Figure 12: Real dependency view between `check_horizontal` and `check_vertical`. The dependence is caused by the shared variable `equal`.

As shown in Figure 13, `tile_filling` depends on the previous border checks for two reasons. First, it uses `equal` to decide whether the tile can be copied directly or whether

a full computation is required. Second, in the uniform-tile case it needs the border value that will be replicated through the tile, which Tareador reports as a dependency through the heap-allocated matrix storage. Therefore, the dependencies in this strategy are mainly due to the shared decision variable `equal` and to the read of the value copied into the tile, rather than to inherent algorithmic dependencies between all the tasks.

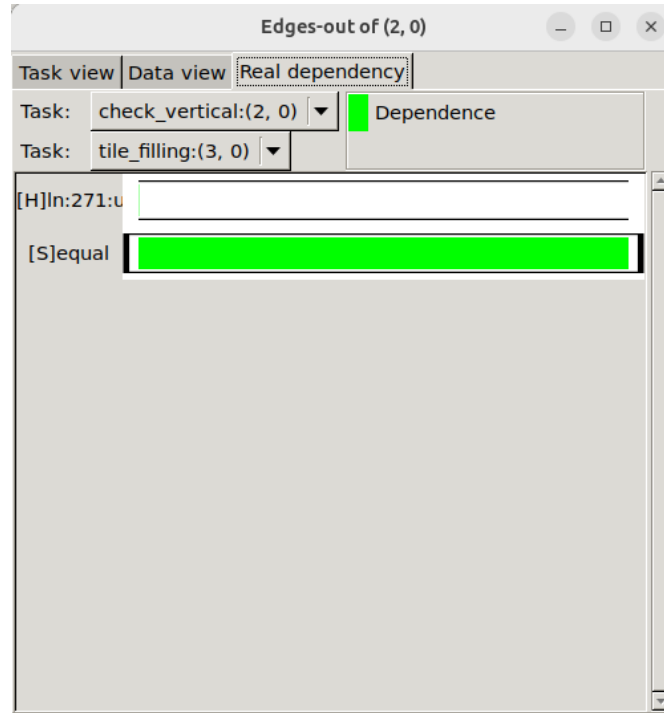


Figure 13: Real dependency view between `check_vertical` and `tile_filling`. Tareador reports dependencies through `equal` and through the heap-allocated matrix storage associated with `M`.

Thus, the fine-grain decomposition introduces data-sharing dependencies mostly because intermediate results are reused through shared state. The control dependence through `equal` is expected in the current formulation, whereas the additional matrix dependence comes from reading the value that is later copied across the tile.

Code Changes

The shared variable `equal` was split into `equal_horizontal` and `equal_vertical`, and the copied value `M[y][x]` was first stored in a private variable `val`.

Original fragment

```
int equal;
```

```

tareador_start_task("check_horizontal");
equal = 1;
...
equal = equal & (...);
tareador_end_task("check_horizontal");

tareador_start_task("check_vertical");
...
equal = equal & (...);
tareador_end_task("check_vertical");

tareador_start_task("tile_filling");
if (equal) {
    ...
    M[py][px] = M[y][x];
}
tareador_end_task("tile_filling");

```

Modified fragment

```

int equal;
int equal_horizontal;
int equal_vertical;

tareador_start_task("check_horizontal");
equal_horizontal = 1;
...
equal_horizontal = equal_horizontal & (...);
tareador_end_task("check_horizontal");

tareador_start_task("check_vertical");
equal_vertical = 1;
...
equal_vertical = equal_vertical & (...);
tareador_end_task("check_vertical");

tareador_start_task("tile_filling");
equal = equal_horizontal & equal_vertical;
if (equal) {
    int val = M[y][x];

```

```
    ...  
    M[py][px] = val;  
}  
tareador_end_task("tile_filling");
```

Task Dependency Graph Without Data-Sharing Dependencies

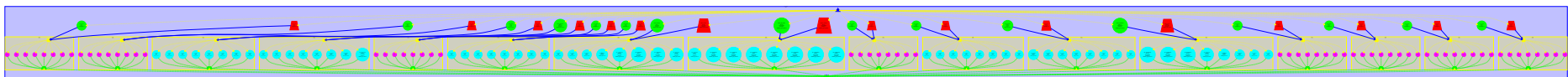
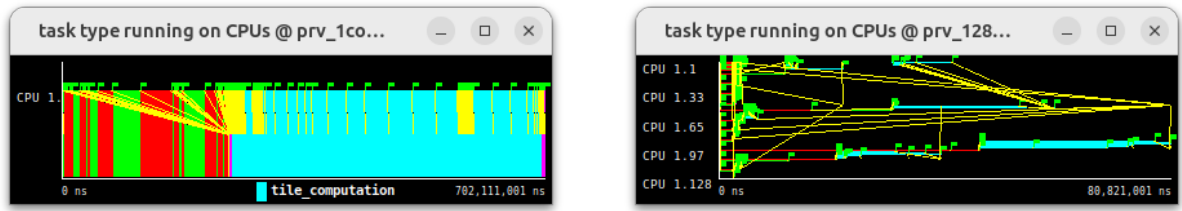


Figure 14: Task dependency graph for the fine-grain strategy after removing the dependencies caused by data sharing.

After removing the data-sharing dependencies caused by `equal` and by the copied tile value, the resulting TDG only reflects the intended control flow of the fine-grain decomposition. In particular, `check_horizontal` and `check_vertical` no longer introduce extra dependencies due to shared state, and `tile_filling` only depends on the outcome of the border checks before spawning the corresponding `tile_copy_borders` or `tile_computation` tasks. This is the structure observed in Figure 14.

It is worth noting that both border-check tasks still conceptually touch the corner value $M[y][x]$. However, this does not create a problematic dependency in this case, because that corner always represents the same Mandelbrot point and is consistently initialized from the same value. Therefore, even if both tasks access that corner-related value, the computation remains correct. If different values could be written there, this would become a real correctness problem, but that is not the case in this implementation. In the modified code, the vertical check avoids reading $M[y][x]$ directly by recomputing the reference value locally, and the copy phase uses a private variable `val`, which is why these additional data-sharing dependencies no longer appear in the graph.

Execution Time



(a) $T_1 = 702,111,001$ ns

(b) $T_{128} = 80,821,001$ ns

Figure 15: Execution-time simulation for the fine-grain strategy.

The simulated sequential execution is $T_1 = 702,111,001$ ns, while the 128-core simulation gives $T_{128} = 80,821,001$ ns, which we use as an approximation of T_∞ .

1.5. Column of tiles parallel strategy

Source code. Submitted file(s): `codes/mandel-seq-iter-tar-column-of-tiles.c`.

Task Dependency Graph

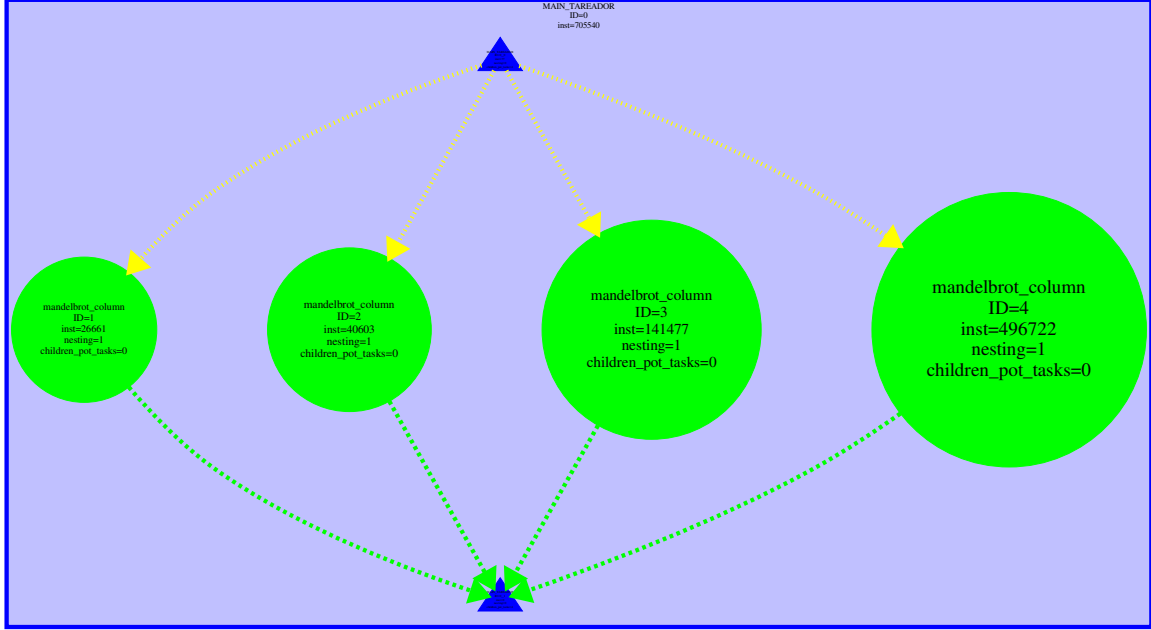


Figure 16: Task dependency graph for the column-of-tiles parallel strategy.

Dependency Analysis

In the column-of-tiles strategy, each task corresponds to one iteration of the outer `x` loop, that is, to one full column of 4 tiles. Therefore, only 4 tasks are created in total. As shown in the Task Dependency Graph, there are no dependencies between `mandelbrot_column` tasks. This is expected, since each task computes a different column of the matrix `M`, and different columns occupy disjoint memory regions. Hence, the tasks do not share tile data and no inter-task ordering is imposed by the Mandelbrot computation itself. The only visible dependencies are the setup dependencies from `MAIN_TAREADOR`, which correspond to the initialization of the input parameters and buffers before task creation.

A strong load imbalance is nevertheless visible. The four column tasks execute very different numbers of instructions, because each task processes four tiles sequentially, and the cost of each tile still depends on the behaviour of `pixel_dwell()` and on whether the tile can be filled directly or must be fully computed. Therefore, even though this strategy

is dependency-free at the task level, its parallelism is limited by the uneven amount of work assigned to each column.

Code Changes

With respect to the original tiled version, the only relevant change is the task granularity: the call to `tareador_start_task()` was moved from inside the nested `y/x` loops to the outer `x` loop, and the task name was changed from `mandelbrot` to `mandelbrot_column`. As a result, each task now processes all the tiles of one column instead of a single tile. The computation performed inside each tile remains unchanged.

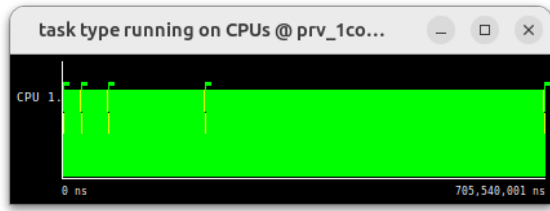
Original fragment

```
for (int y = 0; y < ROWS; y += TILE) {
    for (int x = 0; x < COLS; x += TILE) {
        tareador_start_task("mandelbrot");
        ...
        tareador_end_task("mandelbrot");
    }
}
```

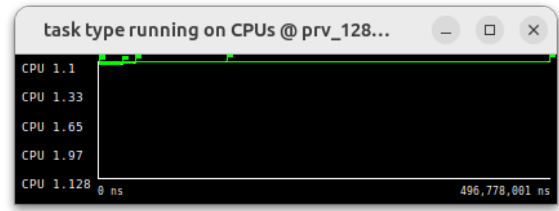
Modified fragment

```
for (int x = 0; x < COLS; x += TILE) {
    tareador_start_task("mandelbrot_column");
    for (int y = 0; y < ROWS; y += TILE) {
        ...
    }
    tareador_end_task("mandelbrot_column");
}
```

Execution Time



(a) $T_1 = 705,540,001$ ns



(b) $T_{128} = 496,778,001$ ns

Figure 17: Execution-time simulation for the column-of-tiles strategy.

The simulated sequential execution is $T_1 = 705,540,001$ ns, while the 128-core simulation gives $T_{128} = 496,778,001$ ns, which we use as an approximation of T_∞ .

2. Recursive Decomposition Strategies

This section presents the recursive parallelization strategies analyzed for the Mandelbrot program. In particular, we study two variants: the *leaf* strategy and the *tree* strategy. In the leaf strategy, tasks are created only when the recursion reaches a final action, that is, when a region can be filled directly or when the base case is reached and the tile must be fully computed. By contrast, in the tree strategy, tasks are created during the recursive process itself, so the recursive decomposition is already exposed in the Task Dependency Graph. In this case, the task is created when the recursion branches, rather than only when the base case is reached, so the work at the bottom of the recursion is executed inside tasks that were already spawned earlier in the recursive tree.

All recursive programs were executed without additional command-line arguments.

2.1. Leaf recursive parallel strategy

Source code. Submitted file(s): `codes/mandel-seq-rec-tar-leaf.c`.

Task Dependency Graph

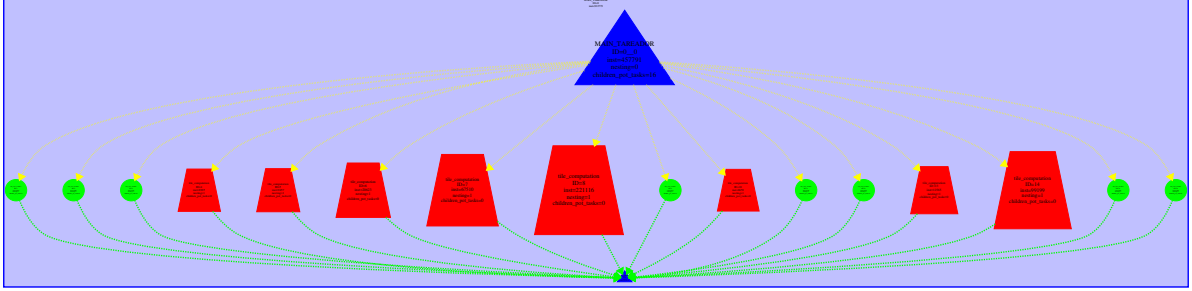


Figure 18: Task dependency graph for the recursive leaf parallel strategy.

Dependency Analysis

In the recursive leaf strategy, tasks are only created at the leaves of the recursion, that is, when the current region can be filled directly (`tile_copy_borders`) or when the recursion reaches the base case `NCols <= TILE` (`tile_computation`). The recursive subdivision itself, together with the horizontal and vertical border checks used to decide which case applies, is executed before spawning those tasks. Therefore, the Task Dependency Graph shows no dependencies between leaf tasks other than the setup relation with `MAIN_TAREADOR`.

However, this apparent lack of dependencies should not be interpreted as high effective parallelism. The graph shows that `MAIN_TAREADOR` absorbs a very large fraction of the total work, while the spawned tasks only correspond to the final leaf actions. This happens because the main recursive execution performs all task creation decisions itself, including the border checks and the recursive calls required to determine whether each leaf becomes a `tile_copy_borders` task or a `tile_computation` task. As a result, a substantial part of the execution remains serialized in the main task, which significantly reduces the amount of useful parallel work exposed by this strategy.

A noticeable load imbalance is also present. Uniform regions generate small `tile_copy_borders` tasks, whereas tiles near the Mandelbrot boundary generate much heavier `tile_computation` tasks. This is reflected in the graph by the large variation in the number of instructions across leaf tasks.

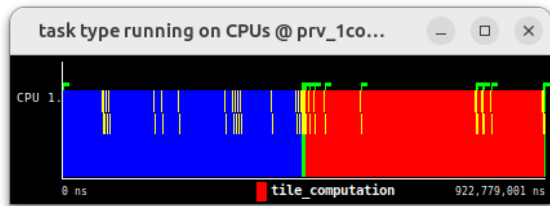
Code Changes

Only the leaf cases were instrumented as tasks. In other words, the recursive calls and the border checks remain inside the sequential recursive flow, and Tareador tasks are only created when the region is either copied directly or fully computed. This matches the definition of the leaf recursive strategy in the statement.

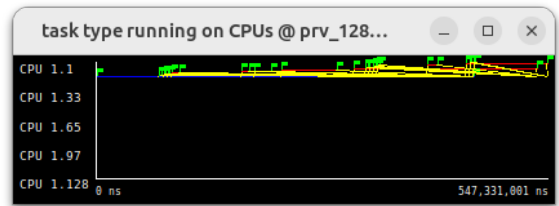
Instrumented placement

```
if (equal) {
    tareador_start_task("tile_copy_borders");
    // fill tile
    tareador_end_task("tile_copy_borders");
} else {
    if (NCols <= TILE) {
        tareador_start_task("tile_computation");
        // compute tile
        tareador_end_task("tile_computation");
    } else {
        // recursive calls
    }
}
```

Time Execution



(a) $T_1 = 922,779,001$ ns



(b) $T_{128} = 547,331,001$ ns

Figure 19: Execution-time simulation for the recursive leaf strategy.

The simulated sequential execution is $T_1 = 922,779,001$ ns, while the 128-core simulation gives $T_{128} = 547,331,001$ ns, which we use as an approximation of T_∞ .

2.2. Tree recursive parallel strategy

Source code. Submitted file(s): `codes/mandel-seq-rec-tar-tree.c` and `codes/mandel-seq-rec-t`. The second file corresponds to the version used to remove data-sharing dependencies in the TDG analysis.

Task Dependency Graph

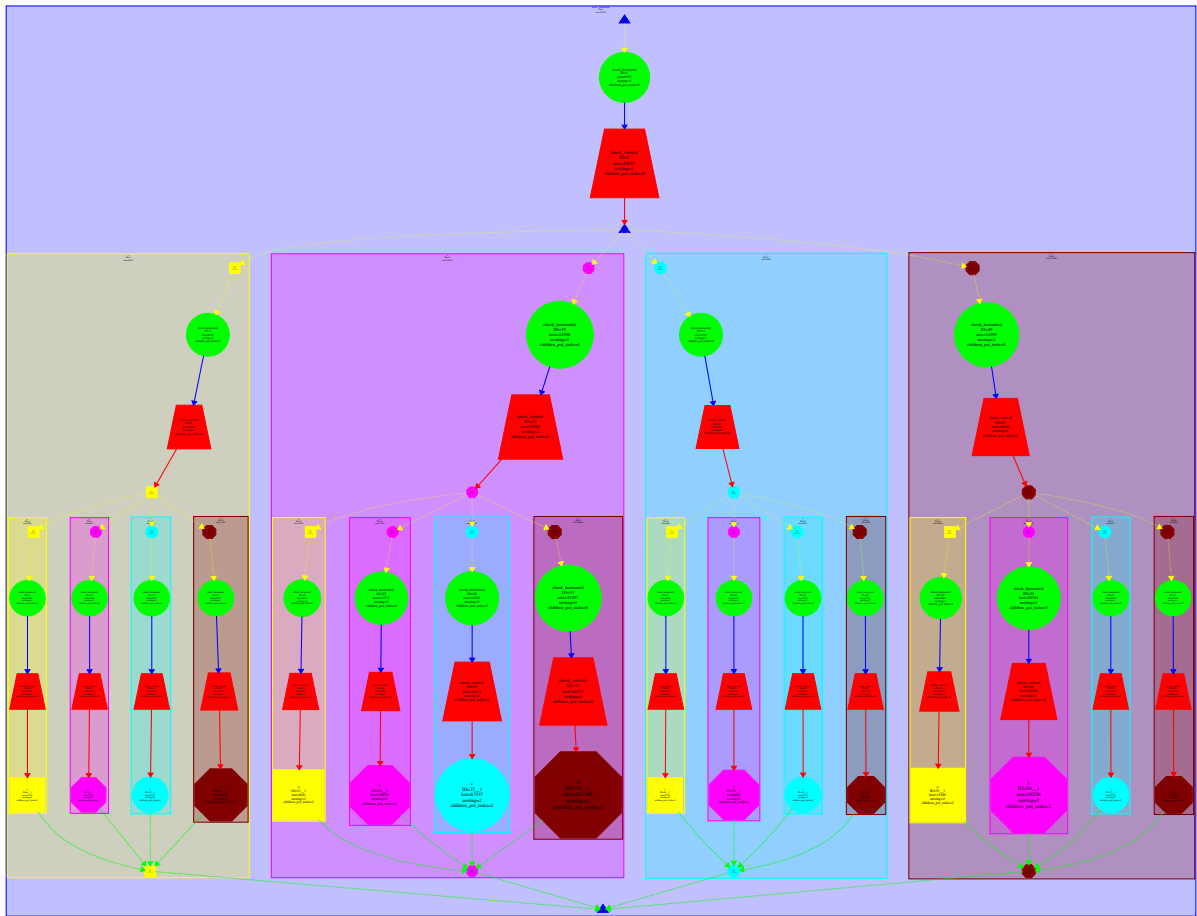


Figure 20: Task dependency graph for the recursive tree parallel strategy.

The TDG has the expected recursive tree structure. At each recursive call, the program first creates the tasks `check_horizontal` and `check_vertical`. If further refinement is needed, it then spawns the child recursive tasks. The graph also shows a clear load imbalance: the upper levels of the recursion contain much heavier tasks, while the checks become progressively smaller as the subregions shrink.

Dependency Analysis

The main explicit dependency in this strategy appears between `check_horizontal` and `check_vertical`, and it is caused by the shared variable `equal`, as shown in Figure 21. The horizontal check updates `equal`, and the vertical check continues from that value before the program decides whether the region is uniform or whether recursion must continue.

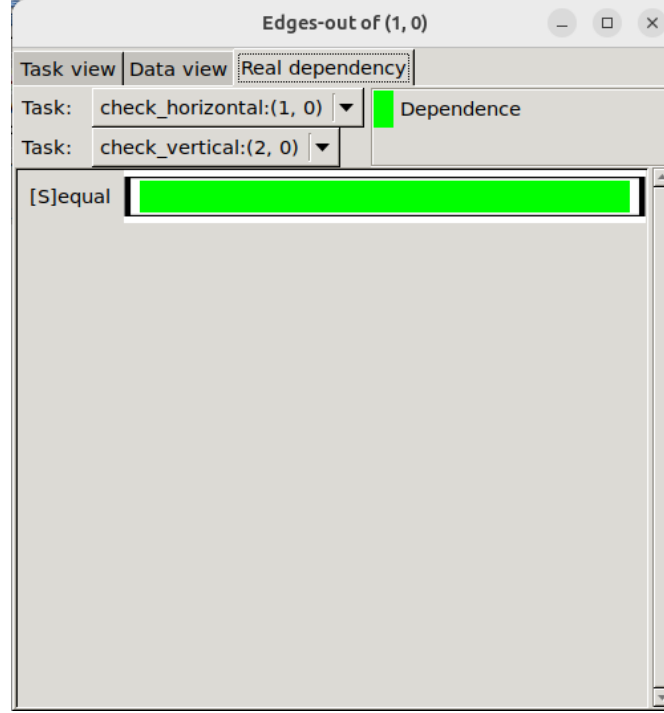


Figure 21: Real dependency view between `check_horizontal` and `check_vertical` in the recursive tree strategy. The dependence is caused by the shared variable `equal`.

Apart from that, sibling recursive tasks do not introduce additional data-sharing dependencies, since each one works on a different subregion of M . However, this does not imply high effective parallelism. At every recursive call, the border checks must be completed before the program can decide whether to stop or which child tasks to spawn. Therefore, the cost of those checks lies on the critical path of the recursion.

Moreover, the same border-checking process is repeated at every recursive call. As a result, the checks performed near the top of the recursion are significantly more expensive than in the iterative strategies, and only become cheaper as the subregions shrink. This creates an important bottleneck, since those larger checks must be completed before deeper recursive tasks can be generated. In this execution, this overhead is especially harmful because useful uniform regions only appear near the bottom of the recursion, so much of the non-parallelizable checking cost is paid before any real benefit is obtained.

Code Changes

In the tree strategy, tasks are created during the recursion itself, not only at the leaves. Therefore, the border checks are instrumented as tasks at every recursive call, and the recursive branches are also spawned as tasks (a, b, c, d, e, f) when the region must be subdivided. This placement matches the idea of the tree strategy, since it exposes the recursive structure of the algorithm directly in the TDG.

Task placement

```
tareador_start_task("check_horizontal");
...
tareador_end_task("check_horizontal");

tareador_start_task("check_vertical");
...
tareador_end_task("check_vertical");

...

tareador_start_task("a");
mandel_tiled_rec(...);
tareador_end_task("a");
...
tareador_start_task("d");
mandel_tiled_rec(...);
tareador_end_task("d");
```

To remove the data-sharing dependency between the two border checks, the original shared variable `equal` was split into `equal_horizontal` and `equal_vertical`. Each check updates its own private value, and both are combined only after the two tasks finish. In this way, `check_horizontal` and `check_vertical` no longer communicate through shared state.

Removal of the equal dependency

```
int equal, equal_horizontal, equal_vertical;

tareador_start_task("check_horizontal");
```



```

equal_horizontal = non_first_call;
...
tareador_end_task("check_horizontal");

tareador_start_task("check_vertical");
equal_vertical = non_first_call;
...
tareador_end_task("check_vertical");

equal = equal_horizontal & equal_vertical;

```

Task Dependency Graph Without Data-Sharing Dependencies

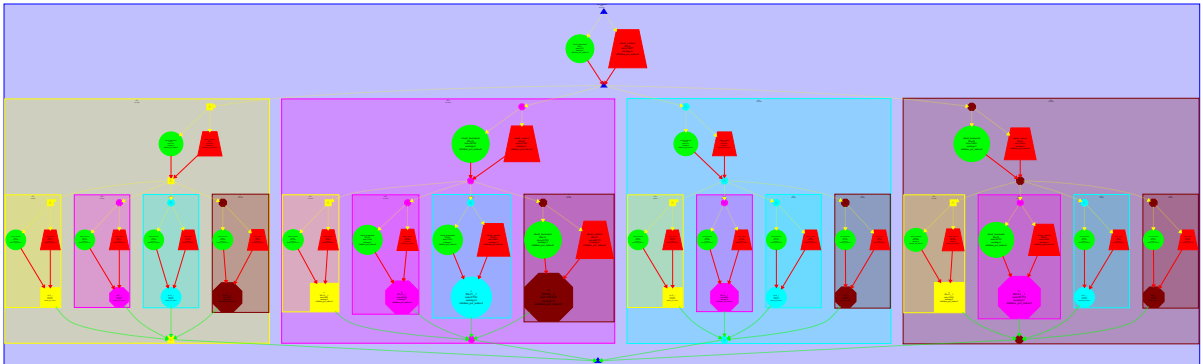
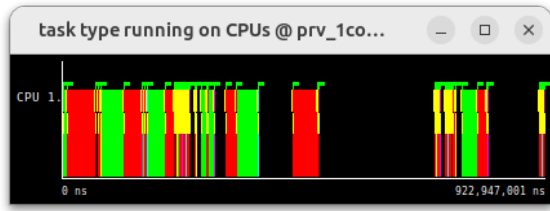


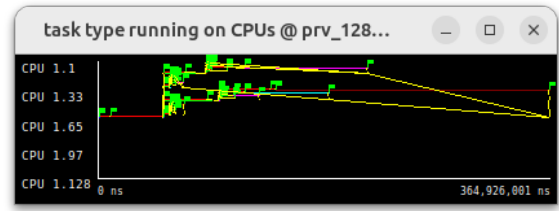
Figure 22: Task dependency graph for the recursive tree strategy after removing the data-sharing dependency caused by `equal`.

After removing the dependency caused by the shared variable `equal`, the extra edge between `check_horizontal` and `check_vertical` disappears, and the TDG reflects only the intended recursive structure of the strategy. Each recursive call first performs the two border-check tasks and, if the region is not uniform, spawns the corresponding child recursive tasks. A clear load imbalance is still visible: the checks near the top of the recursion are much heavier, while the work becomes progressively smaller as the subregions shrink.

Time Execution



(a) $T_1 = 922,947,001$ ns



(b) $T_{128} = 364,926,001$ ns

Figure 23: Execution-time simulation for the recursive tree strategy.

The simulated sequential execution is $T_1 = 922,947,001$ ns, while the 128-core simulation gives $T_{128} = 364,926,001$ ns, which we use as an approximation of T_∞ .

3. Final Strategy Comparison

3.1. Comparison table

Task decomposition	Strategy	Run arguments	T_1	T_∞	Parallelism	Load unbalance: yes/no, why?
Iterative	Original	none	705,540,001	308,750,001	2.29	Yes. Tile costs vary strongly because <code>pixel_dwell()</code> is data-dependent and some tiles can be filled directly while others require full computation.
	Original	-d	729,092,001	310,734,001	2.35	Yes. Same imbalance as the original tiled version. Display introduces only marginal extra cost.
	Original	-h	713,862,001	309,776,001	2.30	Yes. Same imbalance as the original tiled version. Histogram updates add only marginal overhead.
	Fine grain	none	702,111,001	80,821,001	8.69	Yes. <code>tile_copy_borders</code> tasks are very small, while checks and especially <code>tile_computation</code> have variable cost due to <code>pixel_dwell()</code> .
	Column	none	705,540,001	496,778,001	1.42	Yes. Only four coarse tasks are created, one per column, and each column accumulates a very different amount of work.
Recursive	Leaf	none	922,779,001	547,331,001	1.69	Yes. Leaf tasks are uneven, and most recursive exploration remains in the main sequential flow.
	Tree	none	922,947,001	364,926,001	2.53	Yes. Upper recursive checks are heavier and lie on the critical path. Subproblems are also uneven.
Best task decomposition	Fine grain	Highest estimated parallelism and lowest T_∞ . It exposes much more parallel work than the other strategies, although some load imbalance remains.				

Table 1: Summary of the parallel performance for each analyzed strategy.

3.2. Conclusions

The different task decompositions expose very different amounts of parallelism, even though they all implement the same Mandelbrot computation. In all cases, a common source of load imbalance is the data-dependent cost of `pixel_dwell()`, together with the fact that some regions can be filled directly while others require full evaluation. Therefore, the main challenge is not only the presence or absence of dependencies, but also how evenly the work is distributed among tasks.

Among the iterative strategies, the original tiled version is structurally simple and dependency-free at the task level when executed without additional options, but it still suffers from a noticeable load imbalance. The `-d` and `-h` variants show that some dependencies are not inherent to the Mandelbrot algorithm itself, but are introduced by shared display or histogram state. Once these dependencies are removed from Tareador’s analysis, the

underlying parallel structure remains essentially the same as in the original version. The column-of-tiles strategy performs poorly because the task granularity is too coarse: only a few large tasks are created, and the work assigned to each column is very uneven.

The fine-grain iterative strategy provides the best overall result. By splitting the work inside each tile into smaller tasks, it exposes much more parallelism and achieves the lowest estimated T_∞ and the highest parallelism among all analyzed strategies. Although some imbalance remains, especially in the `tile_computation` tasks, this strategy makes much better use of the available parallel work.

Regarding the recursive approaches, the leaf strategy is limited because most of the recursive exploration, including the border checks and the decision of which task to create, remains outside the spawned tasks. As a consequence, a large fraction of the work stays serialized in the main flow. The tree strategy improves on this by exposing the recursive structure itself as tasks, but it still suffers from an important bottleneck: the border checks at the upper levels of the recursion are expensive and lie on the critical path, so the generation of deeper parallel work is delayed.

Overall, the fine-grain iterative decomposition is the most effective strategy among those studied. It offers the best compromise between task granularity and exposed parallelism, while the coarser iterative and recursive strategies leave too much work on the critical path or distribute it too unevenly.